FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate

# BÀI 7: BIẾN ĐỔI DỮ LIỆU

www.poly.edu.vn

# AGENDA

- Độ lệch phân phối
- Biến đổi dương
- Biến đổi âm
- Thực hành

# 1. Độ Lệch Phân Phối

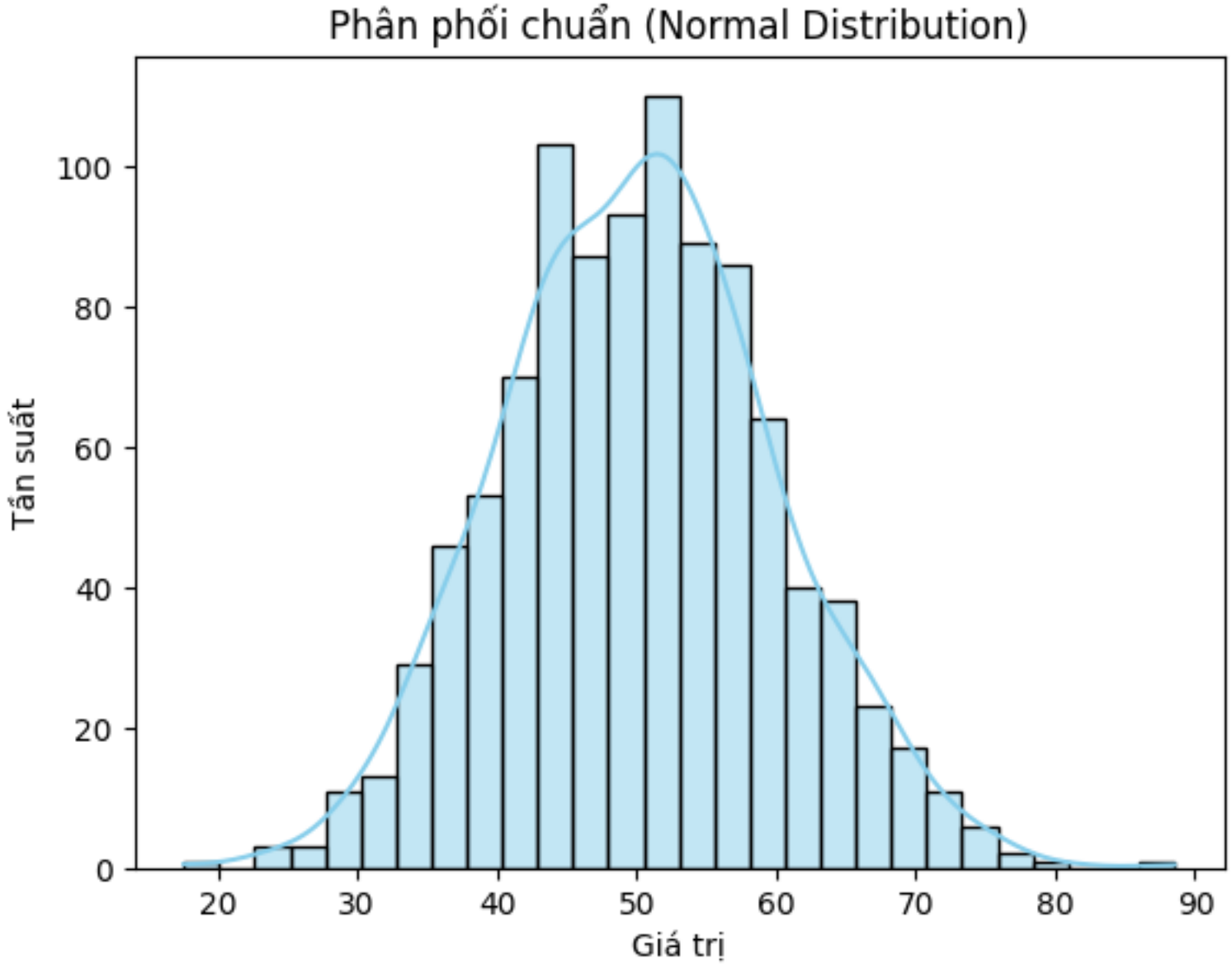
# 1.1 Khái niệm phân phối

- Phân phối dữ liệu mô tả cách các giá trị lan tỏa trong tập dữ liệu.

- Dạng phổ biến nhất: phân phối chuẩn (normal distribution) — cân đối hai bên.

- Tuy nhiên, dữ liệu thực tế thường không chuẩn → xuất hiện độ lệch (skewness).

# 1. ĐỘ LỆCH PHÂN PHỐI



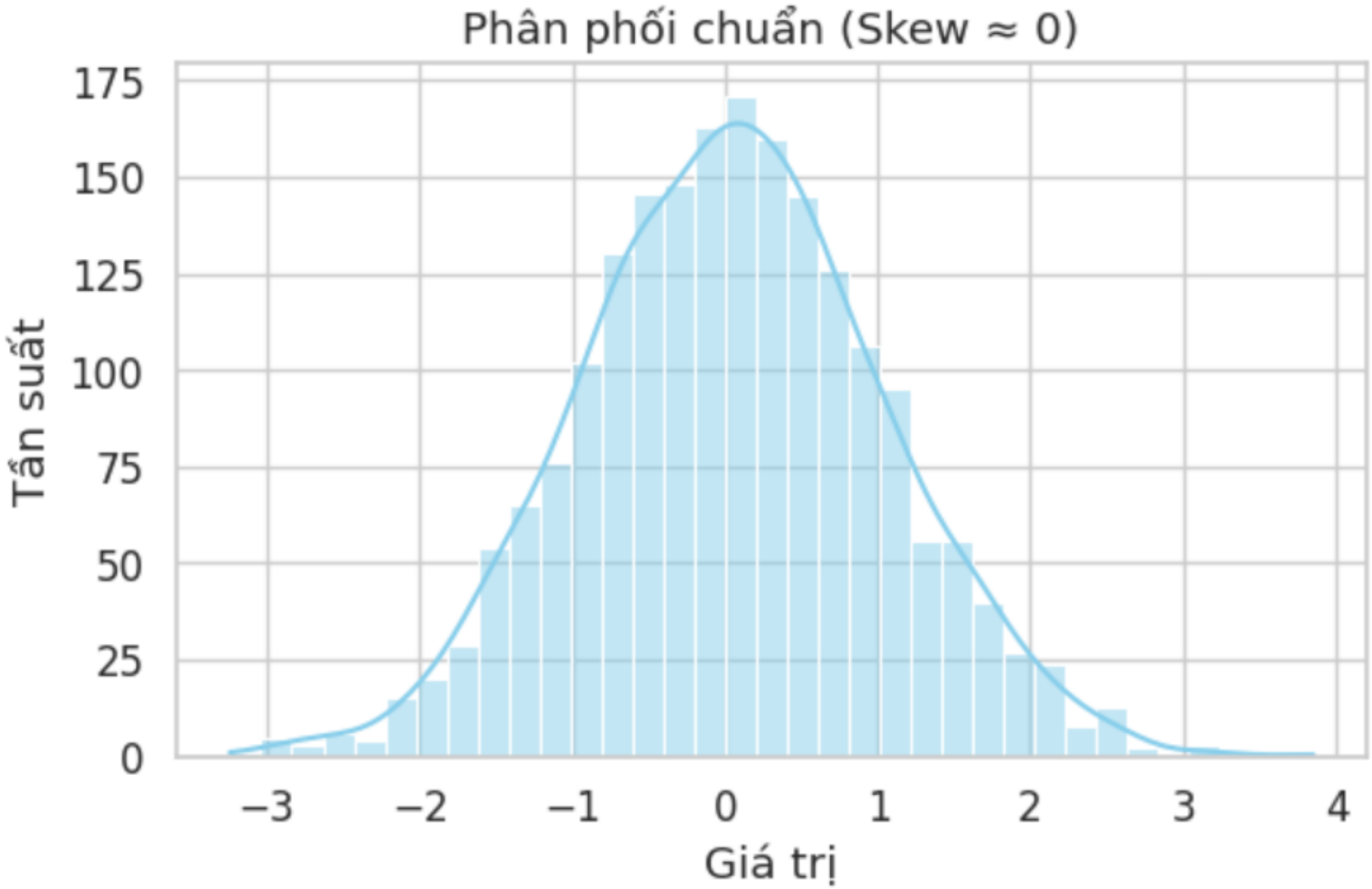
# 1.2 Khái niệm độ lệch

- Độ lệch (skewness) đo mức độ bất đối xứng của phân phối dữ liệu so với phân phối chuẩn-> Giúp nhận biết phía nào của phân phối kéo dài hơn.

- Giá trị skewness:

- ≈ 0 → Phân phối gần chuẩn
- > 0 → Lệch phải (Right-skewed)
- < 0 → Lệch trái (Left-skewed)

# 1. ĐỘ LỆCH PHÂN PHỐI



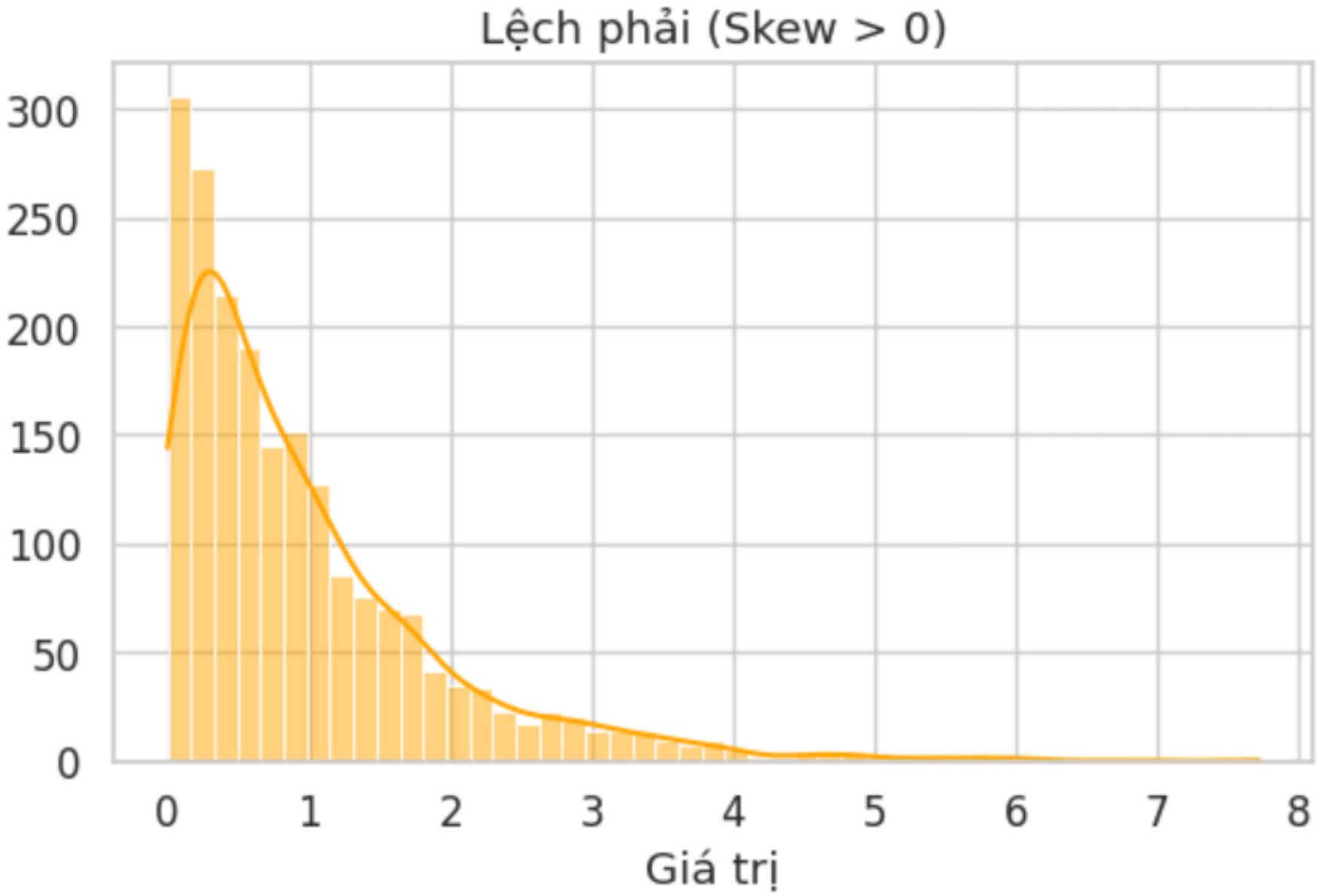
# 1.2 Khái niệm độ lệch

## 1.2.1 Độ lệch dương (Positive Skewness)

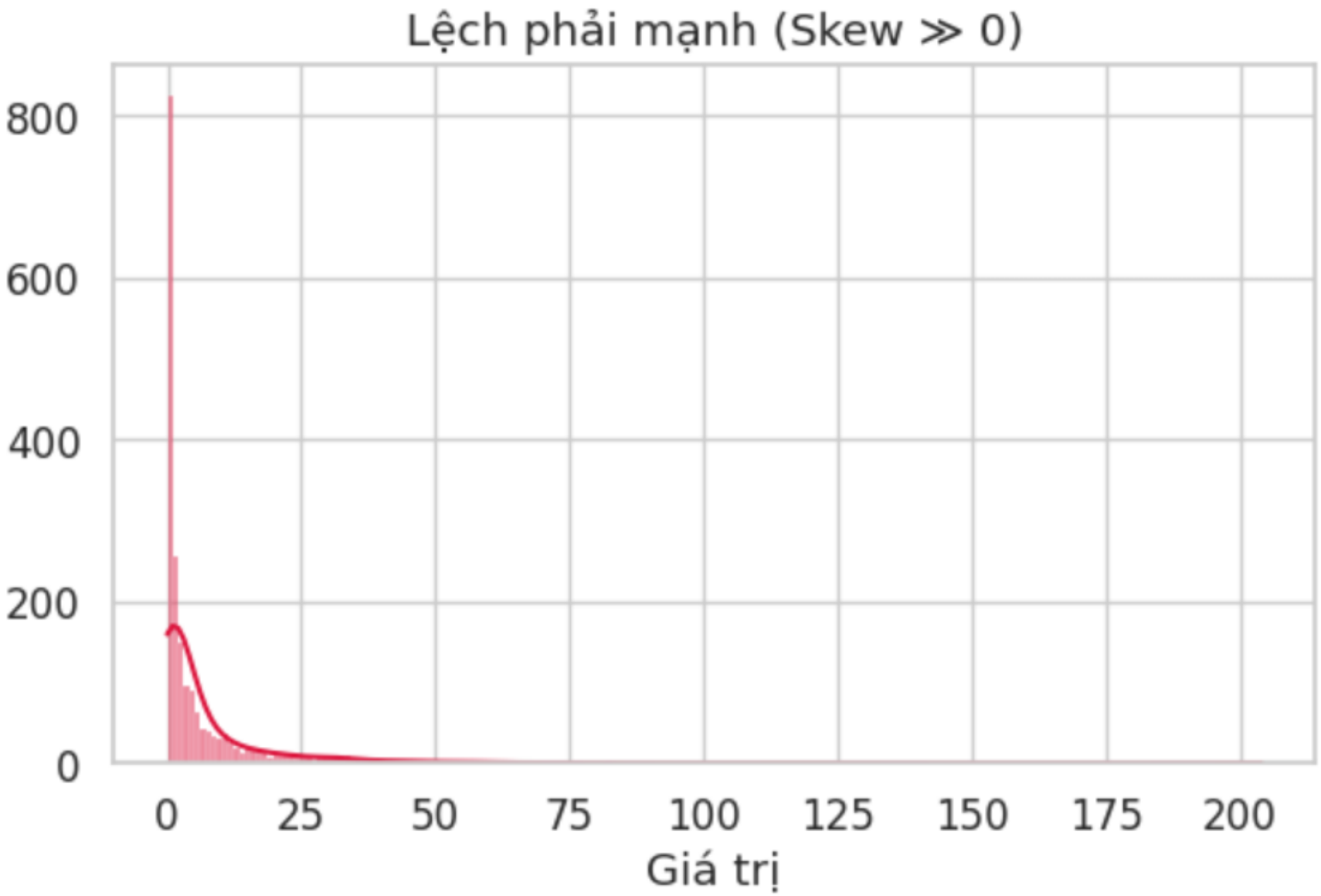
- Đuôi phía bên phải dài hơn.
- Phần lớn giá trị nhỏ, một vài giá trị rất lớn.
- Trung bình > Trung vị > Mode.

Ví dụ thực tế: Giá nhà, thu nhập, doanh thu, thời gian giao hàng.

# 1. ĐỘ LỆCH PHÂN PHỐI



# 1. ĐỘ LỆCH PHÂN PHỐI



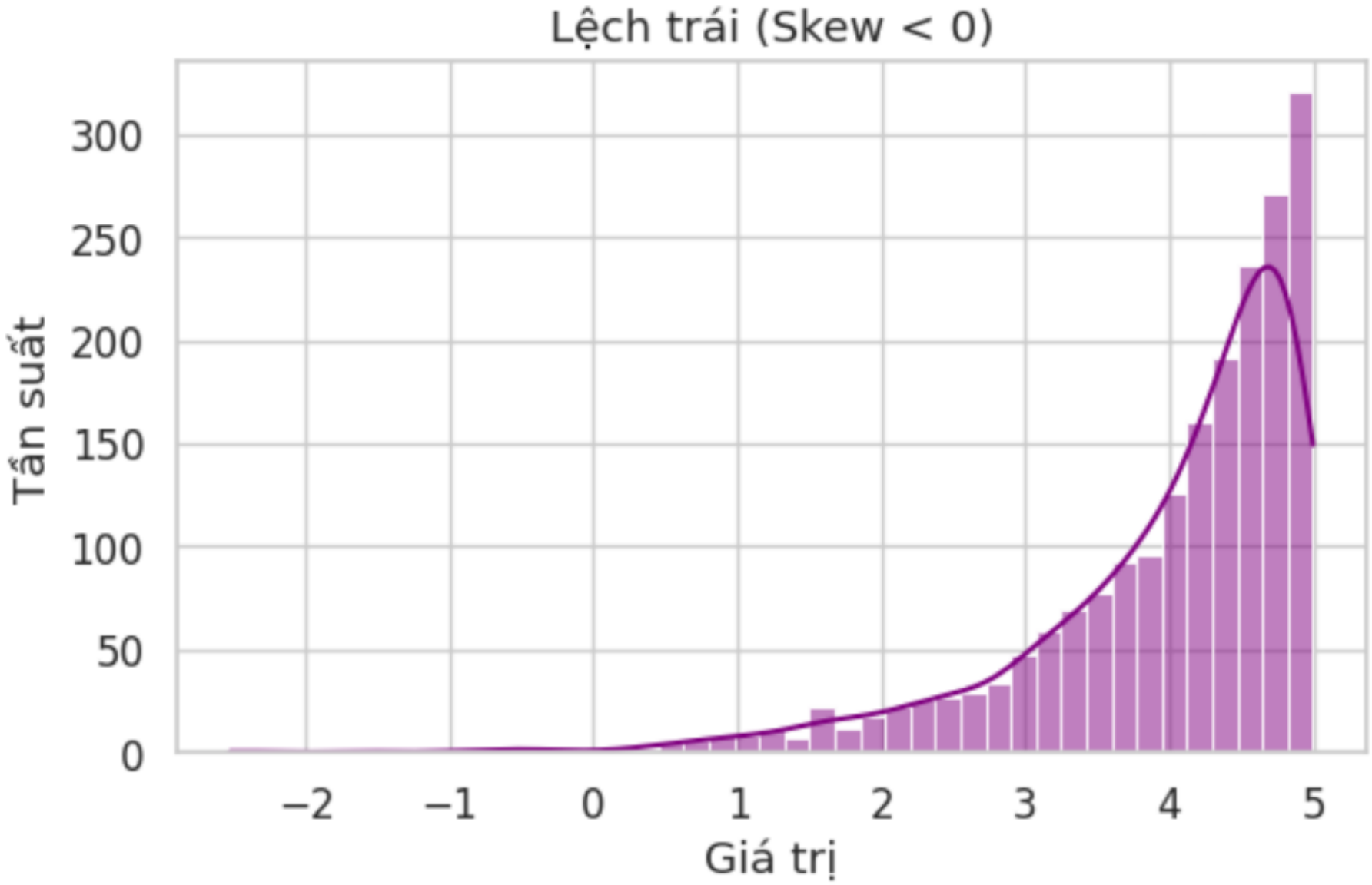
# 1.2 Khái niệm độ lệch

## 1.2.2 Độ lệch âm (Negative Skewness)

- Đuôi phía bên trái dài hơn.
- Phần lớn giá trị lớn, một vài giá trị rất nhỏ.
- Trung bình < Trung vị < Mode.

Ví dụ thực tế: Điểm thi (nhiều điểm cao, ít điểm thấp), tuổi nghỉ hưu (đa số cao, ít người nghỉ sớm).

# 1. ĐỘ LỆCH PHÂN PHỐI



# 1.3 Ảnh hưởng

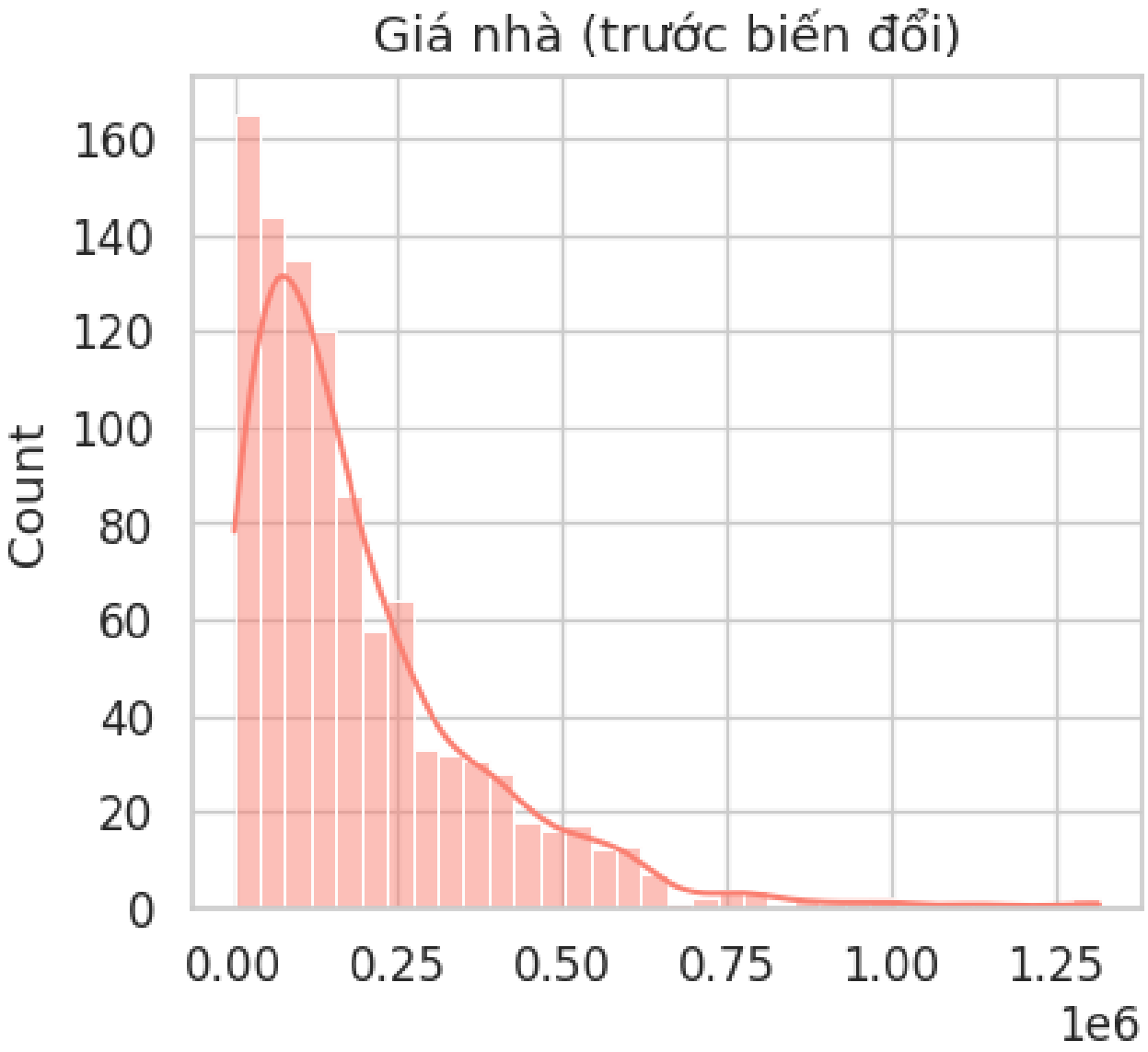
- Một số mô hình giả định dữ liệu gần phân phối chuẩn (Linear Regression, KNN, SVM...).

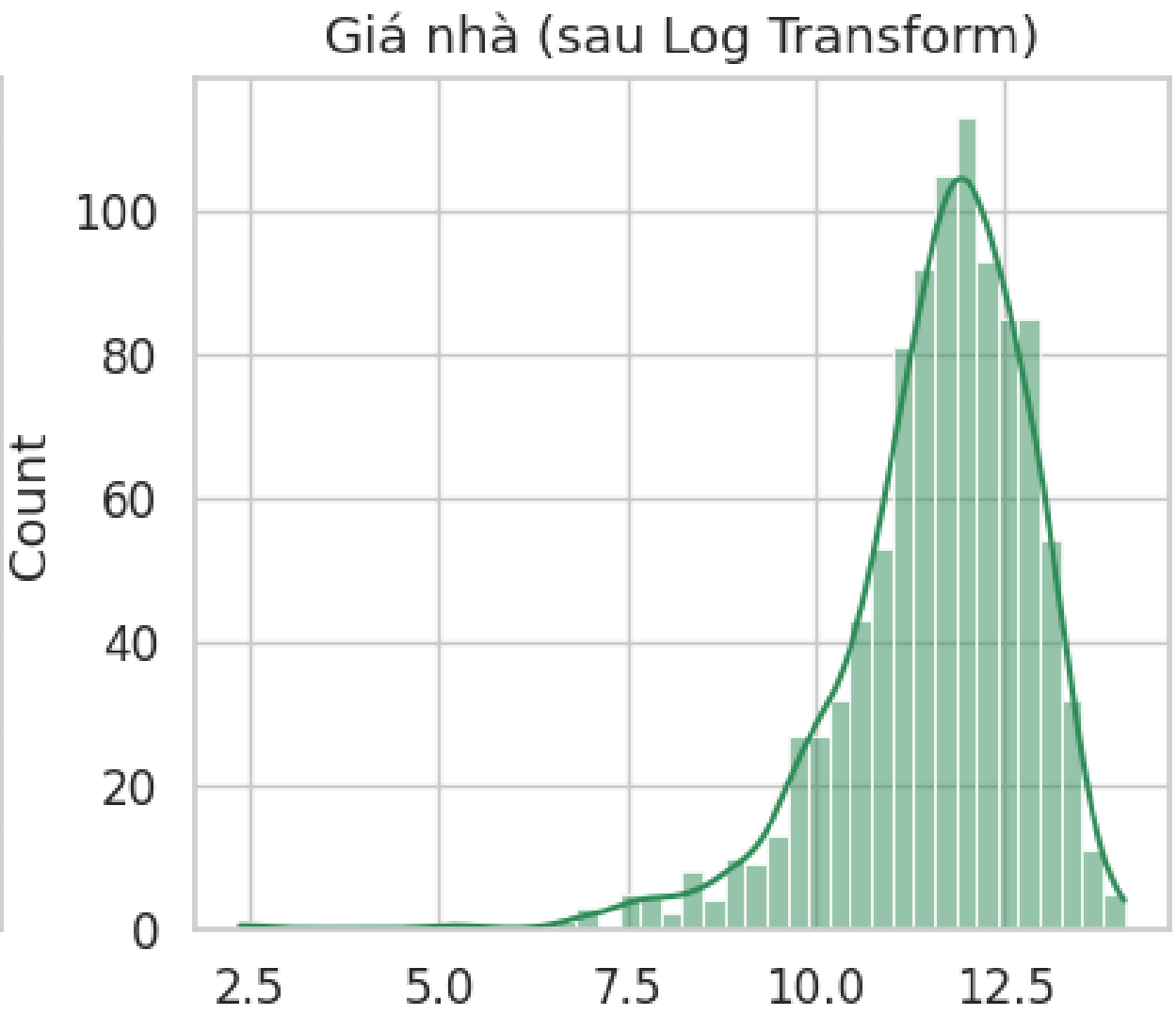
- Dữ liệu lệch mạnh → mô hình:

- Học không ổn định
- Trọng số bị ảnh hưởng
- Giảm hiệu quả dự đoán

-> Cần biến đổi dữ liệu để giảm độ lệch.

# 1. ĐỘ LỆCH PHÂN PHỐI



# 2. Các Phép Biến Đổi

# 2.1 Log Transformation

- Là phép biến đổi dữ liệu bằng logarit.
- Dùng để giảm độ lệch phải (positive skew), giúp phân phối dữ liệu cân đối hơn.
- Thường áp dụng với dữ liệu dương ($X > 0$), ví dụ: giá nhà, thu nhập, doanh thu.

# 2.1 Log Transformation

- Hàm logarit "co" các giá trị lớn → giảm tầm ảnh hưởng của outlier.
- Tác dụng mạnh nhất khi dữ liệu có đuôi dài bên phải.

- Các biến đổi log thường dùng:

- np.log(x) → log tự nhiên (cơ số e)
- np.log10(x) → log cơ số 10
- np.log1p(x) → log(1 + x), dùng khi có giá trị 0.

# 2.1 Log Transformation

- Hạn chế:

- Mất tính trực quan.
- Không áp dụng được với giá trị âm hoặc bằng 0.
- Có thể làm sai lệch quan hệ tuyến tính nếu dùng sai loại dữ liệu.

- Ví dụ sai phổ biến:

- Dùng log cho biến có giá trị âm (như lãi/lỗ).
- Dùng log cho dữ liệu đã chuẩn hóa sẵn → không cần thiết.

## 2.2 Square Transformation

- Là phép biến đổi dữ liệu bằng căn bậc hai.

- Mục tiêu:
  - Giảm độ lệch dương nhẹ đến trung bình (positive skewness).
  - Làm phân phối cân đối hơn.

- Thường dùng cho dữ liệu đếm (count data): số lượt truy cập, số lỗi, số khách hàng, v.v.

## 2.2 Square Transformation

- Hàm căn bậc hai giảm tốc độ tăng của các giá trị lớn, tương tự log nhưng nhẹ hơn.

- Tác động vừa phải → giữ nguyên thứ tự tương đối của dữ liệu.

- Thích hợp khi:

- Dữ liệu có ít giá trị cực đại (outlier vừa phải).
- Không muốn mất tính diễn giải gốc quá nhiều.

# 2.2 Square Transformation

- Hạn chế:

- Chỉ giảm skew dương nhẹ đến trung bình, không đủ mạnh với dữ liệu lệch nặng (đuôi dài).
- Không áp dụng cho dữ liệu âm.
- Không xử lý tốt dữ liệu có giá trị 0 nhiều.

## 2.3 Cubed Transformation

- Là phép biến đổi dữ liệu bằng lũy thừa bậc 3
- Mục đích:

- Giảm độ lệch (skewness) của dữ liệu.
- Làm cho quan hệ giữa biến độc lập và phụ thuộc trở nên tuyến tính hơn.
- Cải thiện hiệu quả mô hình hồi quy, phân tích phương sai hoặc các thuật toán học máy.

- Ứng dụng phổ biến:

- Trong xử lý dữ liệu có giá trị âm (không thể dùng log transformation).
- Khi muốn làm nổi bật sự khác biệt giữa các giá trị lớn.

# 2.3 Cubed Transformation

- Ảnh hưởng đến dữ liệu:

- Giá trị lớn: tăng nhanh hơn (nhấn mạnh chênh lệch).
- Giá trị nhỏ: co lại, giúp dữ liệu cân đối hơn.
- Giá trị âm: vẫn được giữ dấu, không bị loại bỏ.

Là một dạng đặc biệt của Power Transformation (khi $\lambda = 3$ trong mô hình Box-Cox hoặc Yeo–Johnson).

# 2.3 Cubed Transformation

- Hạn chế:

- Khuếch đại outlier: các giá trị lớn bị phóng đại mạnh.
- Giảm khả năng diễn giải: dữ liệu sau biến đổi khó hiểu trực tiếp.
- Không luôn hiệu quả: nếu phân phối dữ liệu gần chuẩn, phép biến đổi có thể làm mất tính tự nhiên.
- Dễ gây mất ổn định số: khi giá trị tuyệt đối quá lớn.

# 2.4 Box-Cox Transformation

- Là phép biến đổi phi tuyến do George Box và David Cox đề xuất (1964).

- Dùng để làm cho dữ liệu gần phân phối chuẩn hơn và ổn định phương sai.

# 2.4 Box-Cox Transformation

- Nguyên lý:

- Biến đổi dữ liệu bằng cách nâng lên lũy thừa $\lambda$, được chọn sao cho dữ liệu sau biến đổi gần chuẩn nhất (thường tối ưu bằng Maximum Likelihood).
- Khi $\lambda$ thay đổi, dạng đường cong của dữ liệu cũng thay đổi.

# 2.4 Box-Cox Transformation

| Giá trị λ | Dạng biến đổi | Ảnh hưởng |
|---|---|---|
| $\lambda = 1$ | Không biến đổi | Giữ nguyên dữ liệu |
| $\lambda = 0.5$ | Căn bậc hai | Giảm skew nhẹ |
| $\lambda = 0$ | Logarithmic | Giảm skew mạnh |
| $\lambda < 0$ | Nghịch đảo hoặc nghịch đảo căn | Xử lý skew cực lớn |

## 2.4 Box-Cox Transformation

- Hạn chế:

- Chỉ có thể áp dụng cho dữ liệu có giá trị dương. Nếu dữ liệu chứa giá trị âm hoặc bằng 0, cần phải chuyển đổi hoặc sử dụng một biến đổi khác.
- Việc tìm giá trị $\lambda$ tối ưu có thể khá phức tạp và đôi khi không rõ ràng, đặc biệt đối với dữ liệu không chuẩn.
- Không phải lúc nào dữ liệu cũng trở thành phân phối chuẩn hoàn toàn.
- Không phải là lựa chọn tốt cho các dữ liệu rời rạc hoặc phân phối nhị thức.

# 2.5 Yeo-Johnson Transformation

- Yeo-Johnson Transformation là một phương pháp biến đổi dữ liệu được phát triển bởi I. Yeo và R. Johnson vào năm 2000.

- Mở rộng của Box-Cox Transformation, áp dụng cho cả dữ liệu âm.

# 2.5 Yeo-Johnson Transformation

- Nguyên lý:

- Khi dữ liệu dương: Yeo-Johnson biến đổi giá trị dữ liệu dương theo cách tương tự như Box-Cox, sử dụng một tham số điều chỉnh gọi $\lambda$

- Khi dữ liệu âm: Yeo-Johnson có một công thức khác để xử lý dữ liệu âm, giúp đưa giá trị âm vào trong phép biến đổi mà vẫn duy trì tính chất của dữ liệu.

# 2.5 Yeo-Johnson Transformation

$$y_i^{(\lambda)} = \begin{cases} ((y_i + 1)^{\lambda} - 1)/\lambda & \text{if } \lambda \neq 0, y \geq 0 \\ \ln(y_i + 1) & \text{if } \lambda = 0, y \geq 0 \\ -((-y_i + 1)^{(2-\lambda)} - 1)/(2 - \lambda) & \text{if } \lambda \neq 2, y < 0 \\ -\ln(-y_i + 1) & \text{if } \lambda = 2, y < 0 \end{cases}$$

# 2.5 Yeo-Johnson Transformation

- Hạn chế:

- Giá trị $\lambda$ tối ưu vẫn có thể là một thử thách, đặc biệt khi áp dụng cho các bộ dữ liệu phức tạp.
- Không phải lúc nào dữ liệu cũng trở thành phân phối chuẩn hoàn toàn.
- Nếu dữ liệu đã gần chuẩn, việc áp dụng Yeo-Johnson có thể không mang lại nhiều lợi ích.

# 2.6 Quantile Transformation

- Là một kỹ thuật biến đổi dữ liệu được sử dụng để chuyển đổi phân phối của dữ liệu về phân phối chuẩn (hoặc phân phối khác mà người dùng lựa chọn) bằng cách sử dụng các phân vị (quantiles).

- Phương pháp này làm cho dữ liệu trở nên đều đặn hơn, giúp cải thiện hiệu quả của các mô hình thống kê hoặc học máy, đặc biệt là khi dữ liệu không có phân phối chuẩn.

# 2.6 Quantile Transformation

- Nguyên lý:

- Xếp dữ liệu theo thứ tự từ nhỏ đến lớn.
- Chia thành các phân vị: 1%, 5%, 10%,...
- Mỗi giá trị gốc được ánh xạ vào một giá trị từ phân phối chuẩn hoặc phân phối mục tiêu dựa trên thứ tự các quantiles.

-> Điều này giúp phân phối của dữ liệu trở nên gần với phân phối chuẩn hoặc phân phối mong muốn.

## 2.6 Quantile Transformation

- Hạn chế:

- Do chỉ dựa vào thứ tự của dữ liệu -> có thể mất đi thông tin về khoảng cách thực tế giữa các giá trị trong bộ dữ liệu.
- Gặp khó khăn với dữ liệu rất lớn vì phải xếp lại và chia dữ liệu.
- Chủ yếu dành cho dữ liệu số liên tục, không thích hợp cho dữ liệu phân loại.
- Dữ liệu có quá nhiều ngoại lai gây ảnh hưởng đến kết quả biến đổi.

# 3. Thực Hành

# 3.1 Nạp và chuẩn bị dữ liệu

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import boxcox
from sklearn.preprocessing import PowerTransformer, QuantileTransformer

plt.style.use("seaborn-v0_8")

df = pd.read_csv("house_data_transform.csv")
df.head()
```

# 3.2 Log Transformation

```
df["price_log"] = np.log(df["price"])

plt.figure(figsize=(10,4))
plt.hist(df["price_log"], bins=40, color='teal')
plt.title("Histogram sau Log Transform")
plt.show()
```

# 3.3 Square Transformation

```
df["price_square"] = df["price"] ** 2

plt.figure(figsize=(10,4))
plt.hist(df["price_square"], bins=40, color='orange')
plt.title("Histogram sau Square Transform (x^2)")
plt.show()
```

# 3.4 Cubed Transformation

```
df["price_cubed"] = df["price"] ** 3

plt.figure(figsize=(10,4))
plt.hist(df["price_cubed"], bins=40, color='purple')
plt.title("Histogram sau Cubed Transform (x^3)")
plt.show()
```

# 3.5 Box-Cox Transformation

```
bc_price, bc_lambda = boxcox(df["price"])
df["price_boxcox"] = bc_price

print("Lambda Box-Cox:", bc_lambda)

plt.figure(figsize=(10,4))
plt.hist(df["price_boxcox"], bins=40, color='green')
plt.title("Histogram sau Box-Cox Transform")
plt.show()
```

# 3.6 Yeo-Johnson Transformation

```
pt = PowerTransformer(method="yeo-johnson")

df["price_yeojohnson"] = pt.fit_transform(df[["price"]])

print("Lambda Yeo-Johnson:", pt.lambdas_)

plt.figure(figsize=(10,4))
plt.hist(df["price_yeojohnson"], bins=40, color='red')
plt.title("Histogram sau Yeo-Johnson Transform")
plt.show()
```

# 3.7 Quantile Transformation - Normal

```
qt_norm = QuantileTransformer(output_distribution="normal", random_state=42)
df["price_quantile_norm"] = qt_norm.fit_transform(df[["price"]])

plt.figure(figsize=(10,4))
plt.hist(df["price_quantile_norm"], bins=40, color='blue')
plt.title("Histogram sau Quantile → Normal")
plt.show()
```

# 3.7 Quantile Transformation - Uniform

```
qt_uni = QuantileTransformer(output_distribution="uniform", random_state=42)
df["price_quantile_uniform"] = qt_uni.fit_transform(df[["price"]])

plt.figure(figsize=(10,4))
plt.hist(df["price_quantile_uniform"], bins=40, color='gold')
plt.title("Histogram sau Quantile → Uniform")
plt.show()
```

FPT Education

# FPT POLYTECHNIC

# THANK YOU